

UNIT IV

FUNDAMENTALS OF JAVA AND PACKAGES

Course Objective: To learn the OOP principles of JAVA and creation of package

Course Outcome: Develop a simple Java Program and can create and import a package.

Bloom's Taxonomy Level: K1, K2

Part A

S.No.	Question	BTL*
1	What are the three core principles of Object-Oriented Programming (OOP) in Java? ● Encapsulation: Bundling of data and methods that operate on the data within a class. ● Inheritance: A mechanism where a new class inherits properties and behaviors (methods) from an existing class. ● Polymorphism: The ability of different objects to be accessed through the same interface, usually achieved by method overriding or overloading.	K1
2	Explain how a Java class is structured, including its main components. A Java class typically consists of: ● Class declaration: Includes class name and modifiers (e.g., public class MyClass). ● Fields (variables): Store the state of an object. ● Methods: Define the behavior of an object. ● Constructors: Initialize object state when a class instance is created.	K1
3	Write a simple Java program that prints "Hello, World!" to the console. <pre>public class HelloWorld { public static void main(String[] args) { System.out.println("Hello, World!"); } }</pre>	K1
4	Describe the finalize method in Java. When is it called? The finalize method is called by the garbage collector before an object is destroyed. It allows the object to clean up resources (like closing files) before it is removed from memory. However, it is now deprecated and is rarely used due to inefficiencies and unpredictability.	K1
5	Define method Methods are functions that operates on instances of classes in which they are defined. Objects can communicate with each other using methods and can call methods in other classes. Just as there are class and instance variable, there are class and instance methods. Instance methods apply and operate on an instance of the class while class methods operate on the class.	K1
6	What is the use of 'this' Keyword? ● this can be used to refer current class instance variable. ● this can be used to invoke current class method (implicitly) ● this() can be used to invoke current class constructor. ● this can be passed as an argument in the method call.	K1

	• this can be passed as argument in the constructor call. • this can be used to return the current class instance from the method.	
7	What gives java it's "write once and run anywhere" nature? All Java programs are compiled into class files that contain byte codes. These byte codes can be run in any platform and hence java is said to be platform independent.	K1
8	Mention why Java is Platform Independent? Java is platform independent because it can run on any platform i.e on windows, mac etc. It contains its own virtual machine which have JRE which runs java program as byte code. There may be different JRE for different platforms. Once it is installed in the system any java application can run on it.	K1
9	Write a simple example of how to add two numbers using a method in Java. <pre> public class AddTwoNumbers { // Method to add two numbers public static int add(int a, int b) { return a + b; } public static void main(String[] args) { // Variables to store numbers int num1 = 10; int num2 = 20; // Call the add method int result = add(num1, num2); // Print the result System.out.println("The sum is: " + result); } } </pre>	K1
10	What are the uses of the keyword 'final'? · The class can be declared as final, if instances or subclasses are not to be created. · The variables are declared as final, value of the variable must be provided at the time of declaration. · The Method can be declared as final indicating that they cannot be overridden by subclasses.	K2
11	Difference between method overloading and method overriding in java.	K1

	<table><tr><th>Overloading</th><th>Overriding</th></tr><tr><td>Whenever same method or Constructor is existing multiple times within a class either with different number of parameter or with different type of parameter or with different order of parameter is known as Overloading.</td><td>Whenever same method name is existing multiple time in both base and derived class with same number of parameter or same type of parameter or same order of parameters is known as Overriding.</td></tr><tr><td>Arguments of method must be different at least arguments.</td><td>Argument of method must be same including order.</td></tr><tr><td>Method signature must be different.</td><td>Method signature must be same.</td></tr><tr><td>Private, static and final methods can be overloaded.</td><td>Private, static and final methods can not be override.</td></tr><tr><td>Access modifiers point of view no restriction.</td><td>Access modifiers point of view not reduced scope of Access modifiers but increased.</td></tr></table>	Overloading	Overriding	Whenever same method or Constructor is existing multiple times within a class either with different number of parameter or with different type of parameter or with different order of parameter is known as Overloading.	Whenever same method name is existing multiple time in both base and derived class with same number of parameter or same type of parameter or same order of parameters is known as Overriding.	Arguments of method must be different at least arguments.	Argument of method must be same including order.	Method signature must be different.	Method signature must be same.	Private, static and final methods can be overloaded.	Private, static and final methods can not be override.	Access modifiers point of view no restriction.	Access modifiers point of view not reduced scope of Access modifiers but increased.	
Overloading	Overriding													
Whenever same method or Constructor is existing multiple times within a class either with different number of parameter or with different type of parameter or with different order of parameter is known as Overloading.	Whenever same method name is existing multiple time in both base and derived class with same number of parameter or same type of parameter or same order of parameters is known as Overriding.													
Arguments of method must be different at least arguments.	Argument of method must be same including order.													
Method signature must be different.	Method signature must be same.													
Private, static and final methods can be overloaded.	Private, static and final methods can not be override.													
Access modifiers point of view no restriction.	Access modifiers point of view not reduced scope of Access modifiers but increased.													
12	List the rules for method overriding. 1. The method must have the same name as in the parent class 2. The method must have the same parameter as in the parent class. 3. There must be an IS-A relationship (inheritance).	K1												
13	Define Inheritance . Give the syntax for the same . Inheritance is a core concept of Object-Oriented Programming (OOP) in Java. It allows one class (subclass or child class) to acquire the properties and behaviors (methods) of another class (superclass or parent class). Inheritance promotes code reuse, modularity, and flexibility by enabling a new class to extend the functionality of an existing class without modifying it. Syntax: class Superclass { // Fields and methods of the parent class } class Subclass extends Superclass { // Fields and methods of the child class }	K1												
14	What is the significance of the super keyword in Java? Provide a scenario where it is used. The super keyword refers to the parent (super) class and can be used to access superclass methods or constructors. Example: class Parent { void display() { System.out.println("Parent method"); } } class Child extends Parent { void display() { super.display(); // Calls the parent class method System.out.println("Child method"); } }	K1												
15	Can you use this () and super () both in a constructor?	K2												

	We cannot have two statements as first statement, so we either need to call super() or this() but not the both. We can't use both the keywords in the constructor. In Java there is a rule that this() and super() must be first statement in the constructor. So we can't use both together in a constructor.	
16	Illustrate how the super class constructor is called . In Java, the superclass constructor is called using the super() keyword. It is used within the subclass's constructor to invoke the constructor of the superclass. This allows for the proper initialization of the superclass part of the object before the subclass adds its own specific fields and behaviors. Key Points: • If no super() is explicitly called, Java automatically inserts a no-argument super() call at the beginning of the subclass constructor. • If the superclass doesn't have a no-argument constructor, the subclass must explicitly call one of the superclass's constructors using super(parameters). The call to super() must be the first statement in the subclass constructor.	K1
17	Give the syntax for calling a super class constructor. <pre>class Superclass { Superclass() { // Constructor code for Superclass } } class Subclass extends Superclass { Subclass() { super(); // Calls Superclass constructor // Constructor code for Subclass } }</pre>	K1
18	What is a package? • Packages contain a set of classes in order to ensure that class names are unique. Packages are containers for classes that are used to compartmentalize the class name space. Packages are stored in a hierarchical manner and are explicitly imported into new class definition. A period is used as separator to enable this.	K1
19	Write a note on import statement? Classes external to a program be imported before they can be used. To import a class the import keyword should be used as given below import The classes in Java are arranged in hierarchical order. The Java library consists of a number of package. These package contain a set of related classes. The whole path of the class must be specified to import a class from the Java library. Example: <pre>import java.util.Scanner; // Imports the Scanner class from the java.util package public class Example { public static void main(String[] args) { Scanner sc = new Scanner(System.in); // Create a Scanner object } }</pre>	K1
20	What are the different levels of access protection in Java, and how do they control visibility of class members? • Public: Accessible from any class. • Protected: Accessible within the same package or subclasses. • Default (no modifier): Accessible within the same package only. • Private: Accessible only within the class where it's defined.	K1

Part B

S.No.	Question	BTL
1.	Explain the three fundamental Object-Oriented Programming (OOP) principles: Encapsulation, Inheritance, and Polymorphism. Provide an example of each in Java code.	K2
2.	Describe the purpose of the this keyword in Java. Write a Java program to demonstrate the use of this in a constructor	K2
3.	Explain with example the various types of inheritance in JAVA	K2
4.	Illustrate method overriding with a Java program that uses the super keyword to call a superclass method from the subclass.	K2
5.	Discuss in detail how to create and implement User defined packages.	K2
6.	Describe the role of access modifiers (public, private, protected, and default) in Java's package system. Write a Java program that demonstrates how classes from one package can be imported into another package.	K2